Q1. List any four key differences between C and C++.

S.No.	Feature	C	C++
1.	Programming Paradigm	• C is a procedural programming language. • It follows a top-down approach.	• C++ is a multi-paradigm language (supports procedural as well as object-oriented). • It follows both top-down and bottom-up approaches.
2.	Classes and Objects	• C does not support classes and objects. • It does not provide data hiding, encapsulation and abstraction.	• C++ supports classes and objects. • It provides data hiding, encapsulation and abstraction.
3.	Function Overloading	• C does not support function overloading. • Two functions cannot have the same name even if their parameters are different.	• C++ supports function overloading. • Multiple functions can have the same name with different parameters.
4.	Standard Library	• C has a limited set of standard library functions (like printf(), scanf(), malloc() etc.).	• C++ has a rich standard Library including STL (Standard Template Library) which provides containers, algorithms, iterators, etc.

Thus, C++ provides more features and flexibility compared to C.

Q2. List and briefly explain key concepts of Object-Oriented Programming.

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects", which can contain data in the form of fields and code in the form of procedures.

The key concepts of OOP are :

① Class and Object :

- Class is a blueprint or template that defines the properties (data) and behaviors (functions) of objects.
- Object is an instance of a class.

Example : Class - Student

Objects - S1, S2, S3

② Encapsulation :

- Encapsulation is the concept of bundling data (variables) and methods (functions) that operate on the data into a single unit (class).
- It also restricts direct access to some of an object's components, which is a way of preventing accidental interference and misuse.

③ Inheritance :

- Inheritance is a mechanism in which one class acquires the properties (data) and behaviors (methods) of another class.
- It promotes code reusability and establishes an "is-a" relationship.
  Example : Dog is an Animal.

④ Polymorphism :

- Polymorphism means "many forms". It allows a single interface to represent different underlying forms (data types).
- It is achieved through function overloading and function overriding.

⑤ Abstraction :

- Abstraction is the concept of showing only essential features of an object and hiding the unnecessary details.
- It helps in reducing complexity and focusing on what an object does instead of how it does it.

Q3. Explain any four disadvantages of conventional (procedure-oriented) programming.

Conventional or procedure-oriented programming (like C) follows a top-down approach where a program is divided into a set of functions. Although it is simple and efficient for small programs, it has several limitations.

The four disadvantages are :

(1) Lack of Modularity

- In procedure-oriented programming, the program is divided into functions, but data and functions are separate.
- Any change in data structure requires changes in many functions that access that data.

Example : If we change the format of a record, all functions using that record must be modified.

(2) Data is not secure

- Global data can be accessed by any function in the program.
- This can lead to accidental modification of data, which may cause unexpected results.

Example : Any function can change the value of a global variable, even unintentionally.

(3) Difficult to maintain and modify

- As the size of the program grows, it becomes difficult to find and fix errors.
- Adding new features or making changes requires understanding the entire program.

Example : In a large program with many functions, a small change may affect many parts.

(4) No code reusability

- Functions in procedural programming cannot be easily reused in other programs.
- There is no concept like inheritance, which allows reuse of existing code.

Example : If we want to use similar data and functions in another program, we have to rewrite the code.

Thus, procedure-oriented programming has limitations in security, modularity, maintainability and reusability, which are overcome by object-oriented programming.

* * *

Q4. Describe the concept of "data abstraction" with respect of OOP.

Data Abstraction is one of the fundamental concepts of Object-Oriented Programming (OOP). It is the process of showing only the essential features of an object and hiding the internal details or implementation from the user.

In other words, data abstraction focuses on WHAT an object does rather than HOW it does it.

Key Points :

- It hides the unnecessary details and shows only the required information to the user.
- It improves the security of data by preventing direct access to internal data members.
- It helps in reducing the complexity of a program.
- It allows the programmer to change the internal implementation without affecting the external code that uses the object.

How it is achieved :

Data abstraction is achieved in C++ using classes and access specifiers (private, protected, public).

- The data members are declared as private (hidden).
- The member functions (methods) are declared as public, which provide access to the data and perform operations on it.

Example :

Consider a class 'BankAccount'. The user can deposit, withdraw or check balance, but the actual balance data is hidden and can be accessed only through member functions.

Conclusion :

Thus, data abstraction helps in creating secure, flexible and easy to maintain programs by hiding the internal details and exposing only the necessary features to the user.

* * *

Q5. Write a CPP program to demonstrate the scope resolution operator to access a global variable when a local variable with the same name exists.

Aim : To write a C++ program to demonstrate the scope resolution operator (::) to access a global variable when a local variable with the same name exists.

Explanation : The scope resolution operator (::) is used to access a global variable when a local variable having the same name is declared inside a function. By using '::' before the variable name, we can refer to the global variable.

Program :

```
#include <iostream>
using namespace std;

int  x = 100;        // Global variable

void display()
{
    int  x = 50;        // Local variable
    cout << "Local x = " << x << endl;       // Accessing local variable
    cout << "Global x = " << ::x << endl;    // Accessing global variable
}

int main()
{
    cout << "Global x in main() = " << ::x << endl;  // Accessing global variable
    display();
    return 0;
}
```

Output :

```
Global x in main() = 100
Local x = 50
Global x = 100
```

Conclusion :

Thus, the scope resolution operator (::) can be used to access the global variable even when a local variable with the same name exists.

Q6. Analyze how namespaces help in avoiding name conflicts in large C++ programs. Explain with code snippet.

Answer :

In large C++ programs, many developers may define classes, functions, variables, or constants with the same names. This can lead to name conflicts. Namespaces provide a solution by creating a separate scope for identifiers. Names inside a namespace are unique to that namespace, so the same name can be used in different namespaces without any conflict.

How namespaces help :

1. They group related identifiers under a single name.
2. They prevent global name pollution.
3. They allow the same name to be used in different namespaces.
4. They make code more modular, readable and easier to maintain.

Example :

```
#include <iostream>
using namespace std;
namespace Math
{
    int value = 100;
    void show()
    {
        cout << "Math::value = " << value << endl;
    }
}
namespace Science
{
    int value = 200;
    void show()
    {
        cout << "Science::value = " << value << endl;
    }
}
int main()
{
    cout << "Global scope value" << endl;
    int value = 300;   // Global variable

    Math::show();      // Calls Math namespace show()
    Science::show();   // Calls Science namespace show()
    cout << "Global value = " << value << endl;

    return 0;
}
```

Explanation :

- Both Math and Science namespaces have a variable named value and a function named show().
- Without namespaces, using the same names would cause a redefinition error.
- By using namespaces, each name is unique within its own scope.
- The scope resolution operator (::) is used to access members of a namespace.

Output :

```
Global scope value
Math::value = 100
Science::value = 200
Global value = 300
```

Conclusion :

Namespaces in C++ effectively solve the problem of name conflicts in large programs by providing separate scopes. They improve code organization, readability and reusability, making them an essential feature in modern C++ development.

Q7. List and explain the uses of Scope Resolution Operator with code snippet of each one.

The scope resolution operator (::) in C++ is used to access a member or a variable that is outside the current scope. It helps to resolve the ambiguity between same names in different scopes.

S.No.	Use	Explanation	Code Snippet

1	To access a global variable when a local variable with the same name exists.	The local variable shadows the global variable. We use :: to access the global variable.	`int x = 10; // global variable`
			`void display() {`
			`  int x = 20; // local variable`
			`  cout << "Local x = " << x << endl;`
			`  cout << "Global x = " << ::x << endl;`
			`}`
			`int main() {`
			`  display();`
			`  return 0;`
			`}`
			// Output
			Local x = 20
			Global x = 10

2	To access a class variable outside the class.	We use :: with class name to access static (class) members.	`class Demo {`
			`  public :`
			`  static int count;`
			`};`
			`int Demo::count = 0; // definition`
			`int main() {`
			`  Demo::count = 5; // access using ::`
			`  cout << Demo::count;`
			`  return 0;`
			`}`
			// Output
			5

3	To define a member function outside the class.	:: is used to specify which class the function belongs to.	`class Add {`
			`  public :`
			`  void sum(int a, int b);`
			`};`
			`void Add::sum(int a, int b) { // definition`
			`  cout << a + b;`
			`}`
			`int main() {`
			`  Add obj;`
			`  obj.sum(3,4); // calling the function`
			`  return 0;`
			`}`
			// Output
			7

4	To access base class members in derived class.	Used to access hidden members of the base class that are overridden.	`class Base {`
			`  public :`
			`  void show() { cout << "Base"; }`
			`};`
			`class Derived : public Base {`
			`  public :`
			`  void show() { cout << "Derived"; }`
			`  void display() { Base::show(); } // access base version`
			`};`
			`int main() {`
			`  Derived d;`
			`  d.display();`
			`  return 0;`
			`}`
			// Output
			Base

5	To access members of a namespace.	Used to access variables, functions or objects defined in a namespace.	`namespace N {`
			`  int value = 100;`
			`  void show() { cout << "Hello"; }`
			`}`
			`int main() {`
			`  cout << N::value << endl; // access variable`
			`  N::show(); // access function`
			`  return 0;`
			`}`
			// Output: 100 Hello

Conclusion: The scope resolution operator (::) is very useful to remove ambiguity and access the required entity from different scopes.

Q8. Write a C++ program to demonstrate default arguments.

Aim : To write a C++ program to demonstrate default arguments.

Explanation :

Default arguments are the values provided in the function definition. If the caller does not pass the value for these parameters, the default values are used automatically.

Program :

```
#include <iostream>
using namespace std;

void display(int a, int b = 10, int c = 20)    // default arguments
{
    cout << " a = " << a << "  b = " << b << " c = " << c << endl;
}

int main()
{
    display(1, 2, 3);      // all arguments passed
    display(4, 5);         // c takes default value
    display(6);            // b and c take default values
    return 0;
}
```

Output :

```
a = 1  b = 2  c = 3
a = 4  b = 5  c = 20
a = 6  b = 10  c = 20
```

Conclusion :

Thus, default arguments allow a function to be called with fewer arguments. If some arguments are omitted, their default values are used automatically.

ooo

Q9. List the Advantages of OOP.

The main advantages of Object-Oriented Programming are as follows :

① **Modularity** : The program is divided into small, self-contained modules called classes. Each class handles its own data and functions, making the program easier to design and manage.

② **Reusability** : Code reusability is achieved through inheritance. Once a class is written, it can be reused in other programs by creating subclasses, reducing duplication of code and development time.

③ **Maintainability** : Since the program is organized into classes, it is easier to modify and maintain. Changes in one class do not affect other classes.

④ **Scalability** : OOP makes it easy to extend the existing system. New classes and objects can be added with minimal changes to the existing code.

⑤ **Data Hiding (Encapsulation)** : It restricts direct access to data and allows access only through member functions. This protects data from unintended modification and improves program security.

⑥ **Abstraction** : It shows only the essential features of an object and hides the unnecessary details. This reduces complexity and helps the programmer to focus on what an object does rather than how it does.

⑦ **Polymorphism** : It allows objects of different classes to be treated as objects of a common base class. It provides flexibility by allowing the same interface but different implementations.

⑧ **Better Productivity** : OOP simplifies the development process due to modularity, reusability and abstraction, resulting in faster development and fewer errors.

⑨ **Real World Modeling** : OOP is close to real-world entities. Objects represent real-world things, making the program more intuitive and easier to understand.

Conclusion : OOP improves the quality of software by making it modular, secure, flexible, reusable and easy to maintain.

* * *

Q10. Explain the basic principles of Object-Oriented Programming with suitable examples.

The basic principles of Object-Oriented Programming (OOP) are the foundations on which OOP is built. They are as follows :

1. **Class and Object :** A class is a blueprint or template that defines properties (data) and behaviors (functions). An object is an instance of a class.
   Example : Class - Car
   Objects - car 1, car 2, car 3

2. **Encapsulation :** Bundling of data and functions into a single unit (class) and restricting direct access to some components of an object.
   Example : Private data members in a class can be accessed using public member functions (getters and setters).

3. **Abstraction :** Hiding the internal implementation details and showing only the essential features to the user.
   Example : A user knows how to drive a car but not how the engine works internally.

4. **Inheritance :** The process of creating a new class from an existing class. The new class inherits the properties and behaviors of the existing class.
   Example : Class - Animal (base class)
   Class - Dog (derived class) inherits from Animal.

5. **Polymorphism :** The ability of an object to take many forms. The same function or operator behaves differently in different situations.
   Example :
   - Function overloading (same function name, different parameters)
   - Function overriding (same function name in base and derived class with different implementation)

6. **Dynamic Binding :** The call to an overridden function is resolved at run time rather than at compile time.
   Example : When a base class pointer refers to a derived class object, the overridden function of the derived class is executed at run time.

Conclusion : These principles together make OOP a powerful programming paradigm. They help in building modular, reusable, flexible and maintainable software.

— ooo —